JAVA 2

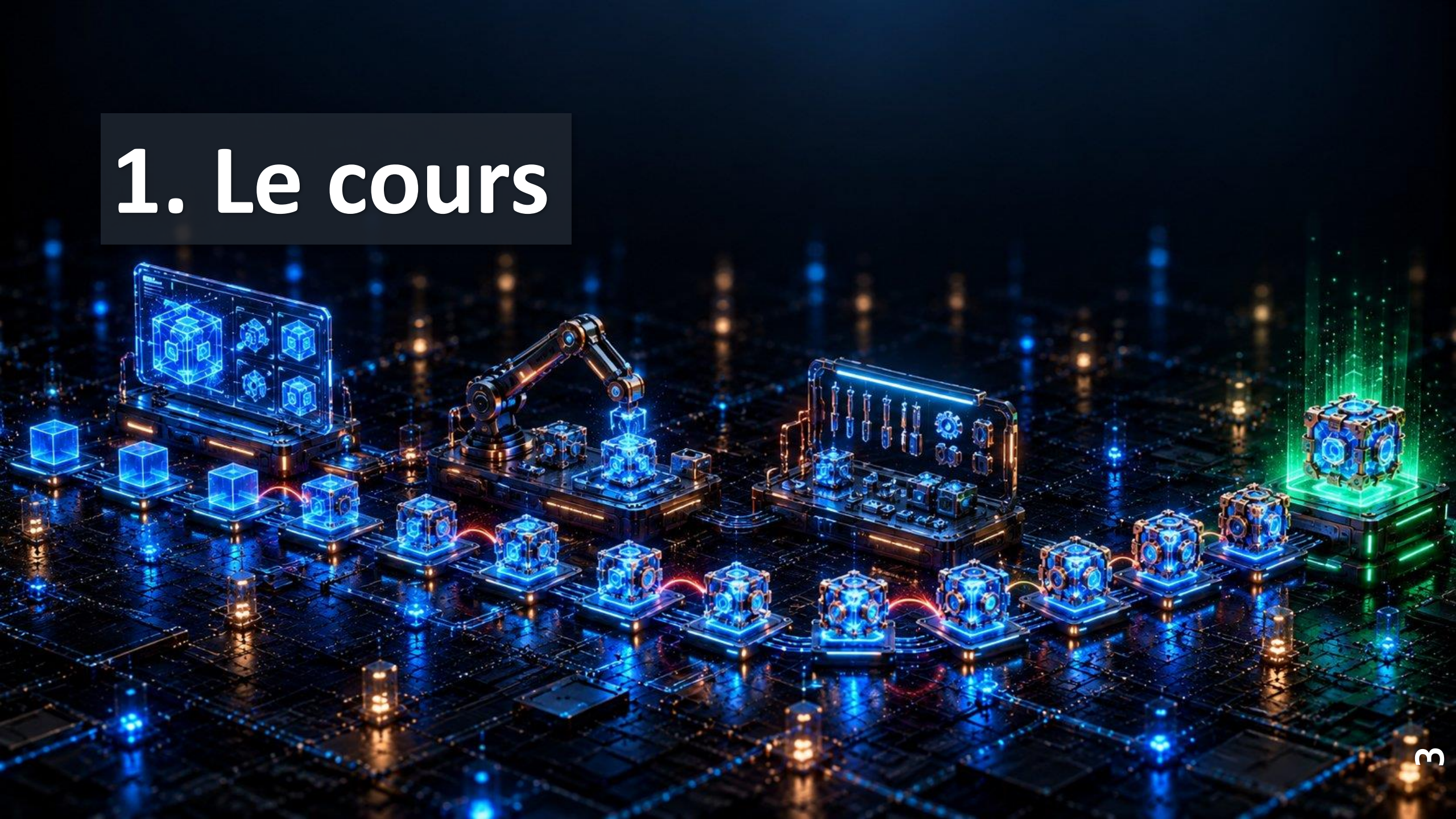
Module INI

Introduction au cours

Au programme

1. Le cours
2. Ma façon de travailler
3. Les mémentos

1. Le cours



Qui suis-je?













EPHEC







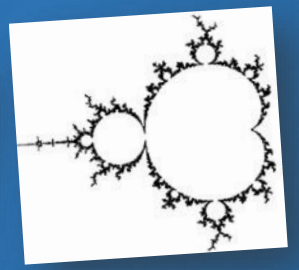




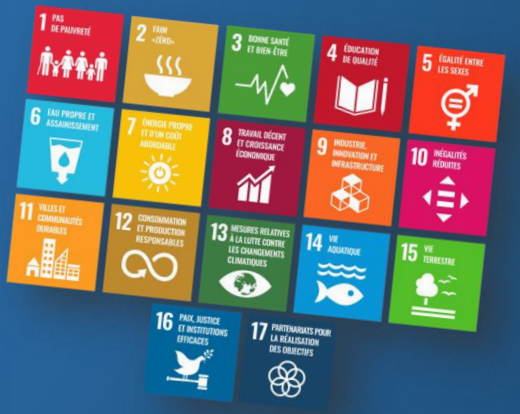












Contenu du cours

- Approfondissement et mise en pratique de ce qui a été vu en C#
- POO, concepts avancés (héritage, types génériques, lambda,…)
- Quelques classes du Framework Java
- Quelques outils (`JUnit`, `JavaFX`…)

- Explications et exercices pratiques
- Travail personnel

- Attention, seul un travail personnel peut vous aider à appréhender ce qui sera vu au cours…
  S'il y a un problème, ne faites pas l'autruche



Les modules du cours

POO	Classes et objets
ITF	Interfaces
GEN	Types génériques
TMP	Dates et durées
PKG	Packages et visibilité
LAM	Classes anonymes, lambdas
TAB	Tableaux
COL	Collections
FX	Interfaces graphiques (`JavaFX`)
ENU	Énumérations et records
EXC	Exceptions
FIC	Fichiers
ABS	Héritage, classes abstraites
JUN	Tests unitaires (`JUnit`)
REC	Exercices récapitulatifs

Ordre donné à titre indicatif

Critères de réussite et acquis d'apprentissage

L'étudiant sera capable [...] de concevoir, d'installer et d'utiliser des objets appropriés à la solution

- Pouvoir définir et implémenter des classes et des objets Java simples
- Pouvoir compléter des classes Java implémentées par un tiers
- Pouvoir utiliser et interagir avec des classes et objets Java existants
- Pouvoir dériver de nouvelles classes par héritage d'une classe Java existante
- Pouvoir utiliser les classes du JDK pour accéder à des fichiers et pour réaliser une application disposant d'une interface graphique simple (`JavaFX`).
- Pouvoir démontrer, au travers d'un exercice simple, sa compréhension du concept et du fonctionnement des expressions lambda introduites par Java 8

L'étudiant sera capable [...] de concevoir et mettre en œuvre une procédure de test partiel et intégré

- Pouvoir implémenter des classes de test simples (`JUnit`,...) sur base d'une classe implémentée par un tiers et/ou de spécifications techniques.
- Pouvoir implémenter des classes et objets Java simples et les classes de test correspondantes.
- Pouvoir, par l'exécution d'une batterie de tests (`JUnit`) implémentée par un tiers, repérer et débugger des problèmes simples dans des classes Java implémentées par un tiers.

Dossier pédagogique

En d'autres termes, vous devez pouvoir…

Concevoir des classes et les utiliser — Comme en C#

Utiliser à bon escient l'héritage

Ecrire et exécuter des cas de test pour vérifier le bon fonctionnement de classes

Utiliser les classes du JDK Java que nous aurons vues au cours: `Fichiers`, `Collections`, `Exceptions`, `Événements`

L'organisation pratique du cours



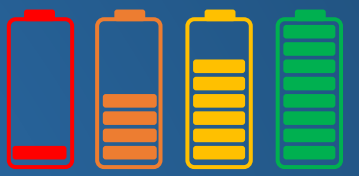
- Le cours comporte 14 séances, du 19 janvier au 18 mai 2027
- Les séances se déroulent de 18 h 00 à (max) 21 h 35, avec une pause de 15 minutes
- L'examen aura lieu le 8 juin 2027
- Outils utilisés : Java, IntelliJ IDEA, Maven, `JUnit`, `JavaFX`
- Pour le détail des versions à installer, voir Moodle

Présence aux séances

Message officiel de la direction de l'EPHEC :

> « En cas de dépassement des 40 % d'absences injustifiées dans une unité d'enseignement, l'étudiant se verra automatiquement désinscrit du cours et non-admis à l'examen. »

… et l'examen?



L'examen se déroule à cahier ouvert (pas de par cœur)

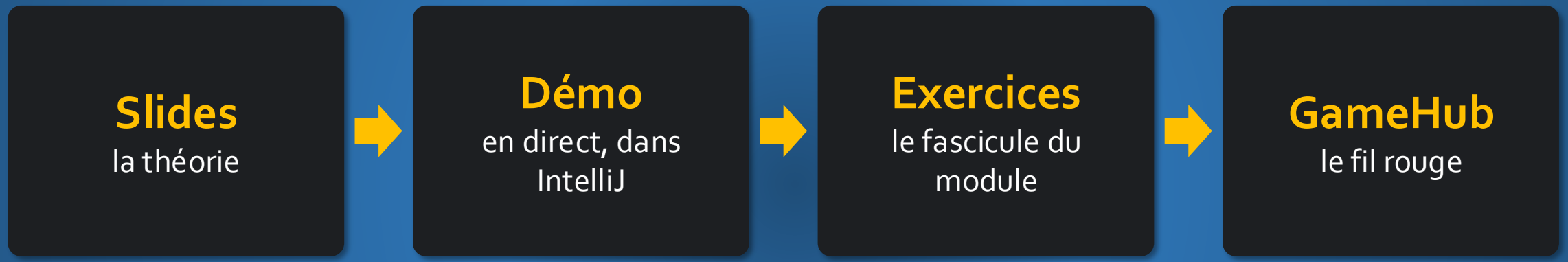
Sur machine, en 2 h 30 : cinq ou six questions, sur les sujets suivants :

1. Ecrire une classe simple (avec `Exception`)
2. Ecrire un type générique
3. Ecrire (et exécuter) une classe de tests pour une classe donnée
4. Ecrire une arborescence d'interfaces/classes
5. Compléter une application graphique pour qu'elle fonctionne
6. … et peut-être une question sur un autre sujet (cela sera confirmé)

2. Ma façon de travailler



Le déroulé d'un module



Une slide de renvoi indique les exercices à faire : `P00.1`, `P00.2`…

Les fascicules d'exercices



Un fascicule par module distribué en PDF

Des exercices numérotés par sujet : `POO.1`, `TAB.3`, `EXC.2`

Le code donne le package dans le projet IntelliJ : `TAB.3` → `tab.e03`

Chaque énoncé suit la même structure

Objectif	Contexte	Énoncé	Spécifications	Vérification	Questions à se poser

Le projet ExercicesJava



Je vous fournis un projet Maven, à ouvrir dans IntelliJ

Vous recevez une archive zip par séance, les solutions seront données après le cours

Des méthodes sont déjà déclarées, à compléter :

```
public int computeAge(int annee) {
    throw new UnsupportedOperationException(
            "computeAge n'est pas encore implementee");
}
```

Les tests fournis

La plupart des exercices ont une classe de test

Vous les lancez vous-même, aussi souvent que vous voulez

Ils ne vérifient pas tout: certaines choses sont laissées à votre choix et non vérifiables par des tests…



Rouge
la réponse n'est pas encore correcte



Vert
la réponse est correcte

Le fil rouge GameHub



GameHub est une plateforme de jeux rudimentaire

Elle se construit au fil des cours

Les étapes numérotées : `G.1`, `G.2`…

Chaque module ajoute une pièce à ce qui a été construit précédemment

GameHub est un projet séparé des exercices

Les tests de module et leurs badges

Un test par module, sur Moodle : réussi à 6 sur 8

Tentatives illimitées ; les questions changent

Une erreur ? Le test dit quoi revoir

Un test réussi : un badge

Pas de note : un encouragement, à votre rythme



Le support imprimable



Les slides sont un appui pour le cours

Je vous distribue également un support de cours qui reprend la théorie rédigée, avec le code

Ce support est un document unique, valable pour tous les modules

3. Les mémentos



Quatre documents de référence

Mémento Exercices		lancer les tests, lire un résultat
Guide IntelliJ	`IDE, ERR`	se servir de l'outil, comprendre une erreur
Mémento technique	`TEC`	comprendre ce qui se passe sous le capot
Mémento Clean Code	`CC`	écrire du code lisible

Sur Moodle, section Documents

Le mémento Exercices

Ce mémento reprend ce qui est commun à tous les fascicules

La numérotation : `EXC.3`, quelle que soit la séance

Mode d'emploi : lancer les tests, et lire leur résultat

Une règle : garder un code qui compile, toujours

Ça ne compile pas ?

Corriger, ou le mettre en commentaire
`Ctrl+/` · `⌘/`

Le guide IntelliJ

Ce guide couvre l'utilisation d'IntelliJ

Les sections `IDE` : installer, créer, exécuter, déboguer

Les entrées `ERR` : les messages d'erreur fréquents

Des captures d'écran à chaque étape

Le mémento technique



Le mémento technique explique ce qu'IntelliJ fait pour vous…

La plateforme Java : JVM, JRE, JDK

Du code source au programme : compilation, exécution

Les outils : Maven, JavaFX, Git, archives zip

Le mémento Clean Code



Ce mémento rassemble les règles qui font la différence entre :
- un code qui fonctionne
- et un bon code.

36 règles, en 7 sections

Noms, méthodes, commentaires, objets, erreurs, tests, classes

Pas un module : des renvois, au fil des exercices

D'après Clean Code, de Robert C. Martin



By Angelacleancoder - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=88628927 - recadrée

Comment lire un renvoi

IDE.14	section 14 du guide IntelliJ
CC-MET.8	règle 8, section « méthodes » du mémento Clean Code

Un renvoi vous dit où chercher, au moment où vous en avez besoin

Quelques liens utiles

Les versions de Java et leur documentation :

docs.oracle.com/en/java/javase/

Un site en français, très complet :

jmdoudoux.fr/java/dej/

En résumé…

Un module = un sujet, avec son code de trois lettres

Slides, démonstration, fascicule d'exercices, GameHub

Des tests pour vérifier vous-même vos réponses

Des mémentos à ouvrir quand un renvoi vous y envoie